

3 | Robot Air Hockey Challenge

This chapter describes the Robot Air Hockey competition that took place from 20 February 2023 to 1 November 2023. It covers the game rules, the motivation behind the event, the various stages of the competition and the outcomes achieved. The following information is derived from the challenge’s website, i.e., <https://air-hockey-challenge.robot-learning.net>.

3.0.1. Air Hockey Game

Air Hockey is a zero sum game in which two opponents compete interacting with a puck on a table and trying to score a goal sending the puck into the opponent’s goal while at the same time preventing the opponent from scoring. The table is rectangular and smooth, with low-friction achieved by air blown from under the surface, designed to mimic the surface of ice. This game is dynamic and the puck reaches high velocities that require the players to have quick reflexes and strategic positioning.

3.0.2. Motivation

This competition was organized to *close the sim-to-real gap*, i.e., the differences and challenges that arise when transferring a robotic system trained in a simulated environment to the real world. In fact, when transferring the robotic system to the real world we have to deal with

- disturbances
- observation noise
- safety
- model mismatch
- delays
- actuator limitations
- physical feasibility
- limited real-world interaction

The Robot Air Hockey Challenge provides a platform for researchers in the field of robot learning to interact with each other on a realistic robotic task. In this competition team had to design and build their air hockey agents, competing against each other in different

subtasks (in simulation) and finally in an entire game (both in simulation and in the real world).

3.1. Challenge organization

The challenge consists in three simulated stages: Warm Up, Qualifying and Tournament. Each stage consists of different tasks required for robot air hockey. Apart from executing the given tasks the robots need to be robust and capable of adapting to changes. Except from the Warm Up stage the submitted agents are evaluated in modified simulation environments to mimic the sim-to-real gap.

The first three teams are able to deploy their approach on the real robot and compete in a real world scenario.

3.1.1. Warm-Up Stage

In the Warm-Up stage the participants were given an ideal environment with a 3-degree-of-freedom robot. This stage aimed to familiarize with the given tasks, the environment and the application programming interface.

The tasks given at this point were two:

- **Hit:** The puck is randomly initialized on the left side of the table. The initial velocity is zero. The objective is to hit the puck to score a goal as fast as possible.
- **Defend:** The puck is randomly initialized on the right side of the table with a random velocity heading left. The objective is to stop the puck on the right side of the table and prevent it from getting scored.

3.1.2. Qualifying Stage

In this stage the participants were given an environment exposing an interface to control a general-purpose robot, the KUKA-iiwa14 LBR. At this point the submitted agents to the challenge's cloud evaluator were simulated with a modified version of the environment handed to the participant teams in order to simulate different types of real world problems. These modifications included disturbances, observation noise, loss of tracking, model mismatch, and imperfect tracking controller.

Each team could evaluate its solution once per task per day. Each evaluation was conducted with 1000 episodes, equivalently 2.8 hours of real-world experiments. Based on the evaluation metric, the agents were categorized into three levels:

- Deployable
- Improvable
- Nondeployable.

The given tasks at this stage were three:

- **Hit:** The opponent moves in a predictable pattern. The puck is randomly initialized with a small velocity. The objective is to score the goal as many times as possible. The episode ends when the puck is scored or it bounces back. The episodes terminates with *success* if the puck is scored.
- **Defend:** The puck is randomly initialized on the right side of the table with a random velocity heading left. The objective is to stop the puck on the right side of the table and prevent it from getting scored. The episode terminates if the puck is returned to the opponent's side or scored or the puck speed drops below the threshold. The episode terminates with *success* if the puck is in the range where hits can be made and the longitudinal speed is below the threshold.
- **Prepare:** The puck is initialized close to the table's boundary and is unsuitable for hitting. The task is to control the puck to move it into a good hit position. The puck is not allowed to cross the middle line. The episode terminates if the puck crosses the middle line that connects the middle points of two goals, or the puck is on the opponent's side of the table. The episode terminates with success if the puck is in the range where it can be made and the longitudinal speed is below the threshold.

3.1.3. Tournament Stage

Only the participant teams who submitted agents that were categorized as Deployable or Improvable were qualified to participate in the tournament stage. The maximum number of teams is 16, which were determined based on the ranking of *success rate* in the qualifying stage. In this stage each team had to develop an agent able to play the whole game. A hard coded baseline agent was provided to test and validate the agents before the actual competition started.

The tournament stage was divided into two rounds. After the first rounds teams had two weeks to adjust and improve their agent. Teams were awarded three points if they won a match, otherwise one point if they drew and zero if they lost. The final ranking of the tournament was determined by the total score of the two rounds.

Game Rules

- A game is 15 minutes in length, i.e. 45000 steps.
- The first player to accumulate seven (7) points wins the game. If no player accumulates seven points within the designated competition time, the player with the most points wins.
- To score a point, the puck must fully enter the goal. Rebounds or pucks that get stuck halfway in do not count as a point.
- When a player makes a goal, the other player serves the puck next.
- A player may only strike the puck when it is on their side of the centerline.
- Mallets may not cross the center line when striking the puck.

- When the puck stops in the middle of the table (20cm width), the puck is reset randomly to one player’s side
- “Topping” the puck is not allowed. This means players cannot lift their mallet and place it over the puck to hold it in place.
- Touching the puck with other parts of the robot is not allowed.
- Once the puck is on a certain player’s side of the center line, the player has 15 seconds, i.e. 750 steps, to hit the puck back across the center line. Otherwise a foul is committed and the opponent receives possession of the puck.

3.2. Framework

The Air Hockey Challenge is built upon MushroomRL [3], a Reinforcement Learning Library. The general framework of challenge consists of two key components, i.e., *Agent* and *AirHockeyChallengeWrapper*. A schema of the framework can be seen in Figure 3.1. Participants had to implement the *Agent* component.

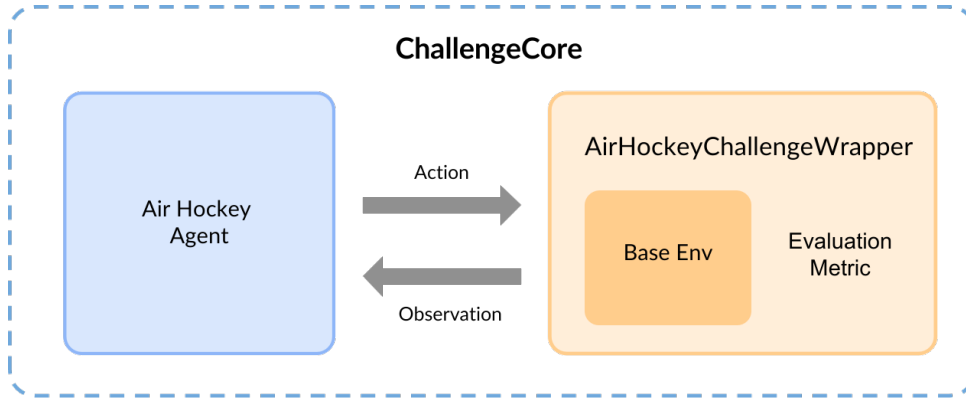


Figure 3.1: Challenge framework.

3.2.1. Environments

The *AirHockeyChallengeWrapper* component is a wrapper for the *Base env* and processes the necessary information for the challenge evaluation.

The simulated environment tries to mimic the real-world setup. The robot is controlled by a *Tracking controller*, a Feedforward-PD controller, which sends the torque command τ_{cmd} to the robot.

$$\tau_{cmd} = M(q)\ddot{q}_d + c(q, \dot{q}) + g(q) + K_p(q_d - q) + K_d(\dot{q}_d - \dot{q})$$

The *Trajectory interpolator*, by default a cubic polynomial, is used to interpolate the trajectory points between two consecutive commands. A *Joint safety limiter* is used to avoid executing commands that exceed the position or velocity limits.

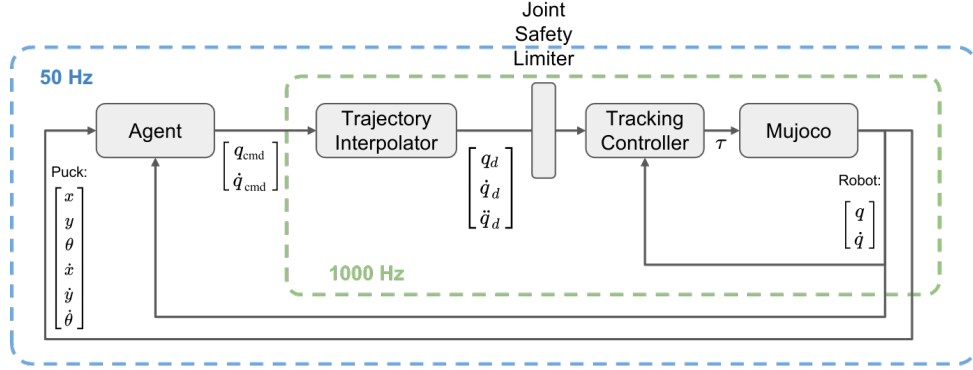


Figure 3.2: Control loop.

The framework provides a flexible interface for commanding the robot in which the trajectory interpolation order can be specified:

- Cubic interpolation. The action command contains the desired [position, velocity]. A cubic polynomial is used to interpolate the intermediate steps.
- Linear interpolation. The action command contains the desired [position]. A linear polynomial is used to interpolate the intermediate steps.
- Quadratic interpolation. The action command contains the desired [position]. A quadratic function uses the previous position, velocity and the desired position to interpolate the intermediate steps.
- Quartic interpolation. The action command contains the desired [position, velocity]. A quartic function uses the previous position, velocity and the desired position, velocity to interpolate the intermediate steps.
- Quintic interpolation. The action command contains the desired [position, velocity, acceleration]. A quintic function is computed by the previous position, velocity, acceleration and the desired position, velocity and acceleration to interpolate the intermediate steps.
- Linear interpolation in position and velocity. The action command contains the desired [position, velocity]. The position and velocity will both be linearly interpolated. The acceleration is computed based on the derivative of the velocity. This interpolation is not proper, but it is useful to avoid oscillatory in the interpolation.
- None. The action can be a complete trajectory between each action step. At each step, the trajectory command should include desired [position, velocity, acceleration]

The air hockey table and its dimensions are illustrated in Figure 3.3

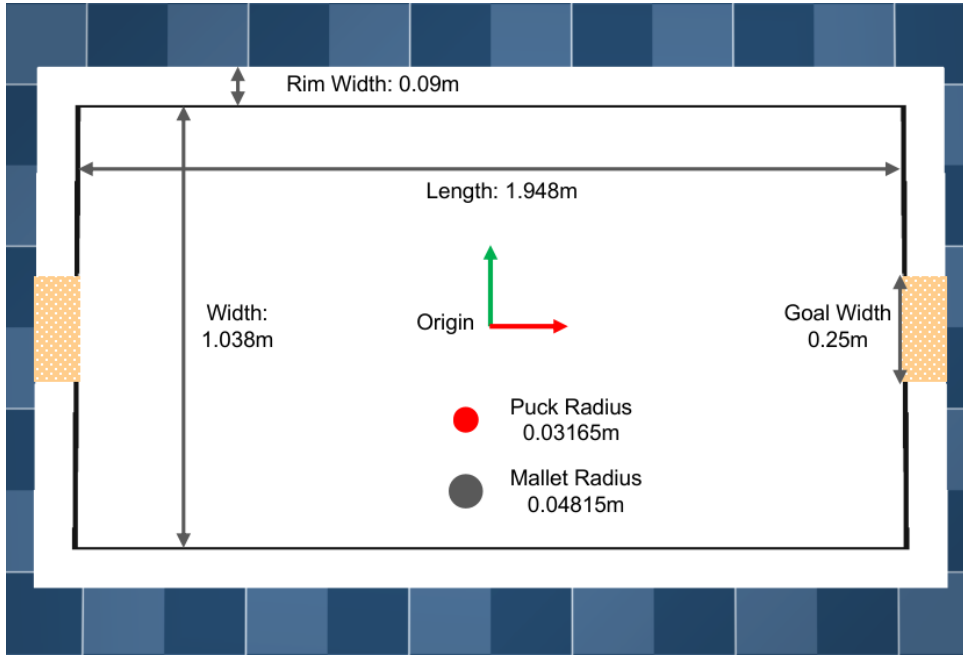


Figure 3.3: The air hockey table.

3.2.2. Robot models

Planar robot - 3 degrees of freedom

During the Warm up stage the participants were provided a 3 DoF planar robot to understand how to use the framework. The robot is set such that the end-effector remains at the same height of the table surface. In simulation, the collision between the robot and the table are discarded. The base of the planar robot is located at $[-1.51, 0., -0.1]$, the orientation is aligned with the world's frame, as illustrated in Figure 3.4

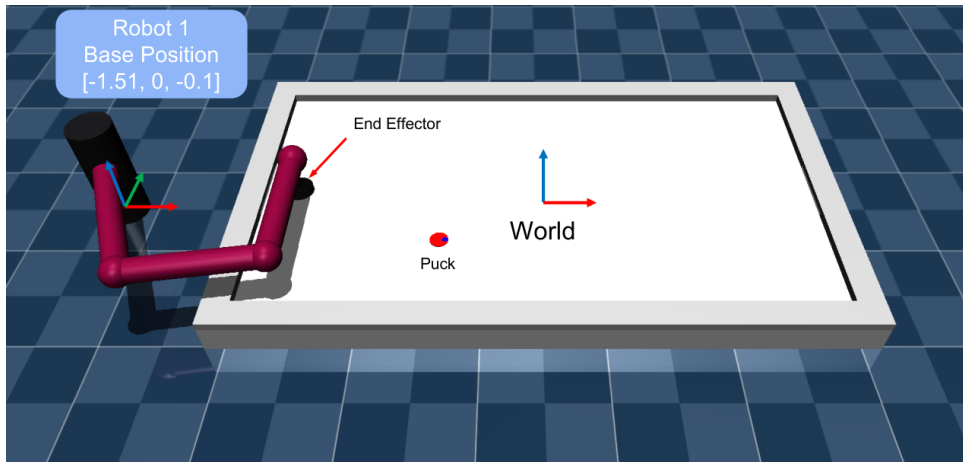


Figure 3.4: Planar robot base frame.

Below the specification of the planar robot:

- **Position upper limit (rad):** [2.967060, 1.8, 2.094395]
- **Position lower limit (rad):** [-2.967060, -1.8, -2.094395]
- **Velocity limit (rad/s):** +/- [1.570796, 1.570796, 2.094395]
- **Link length (m):** [0.55, 0.44, 0.44]
- **Initial position (rad):** [-1.156, 1.300, 1.443]
- **Initial velocity (rad/s)** [0, 0, 0]

KUKA iiwa14 LBR robot

The KUKA iiwa14 LBR robot is a general purpose manipulator with 7 degrees of freedom. A universal joint on the end-effector is added to increase the robot's flexibility. The universal joint is a passive joint that adapts the joint position based on contacts. In simulation, the collision between the robot and the table are discarded, requiring the agent to keep the mallet at an appropriate height.

We define the End-Effector as the tip of the extension rod before the universal joint. The end-effector's position can be fully determined by the robot's joint position and forward kinematics. The base position of the robot is depicted in Figure 3.5

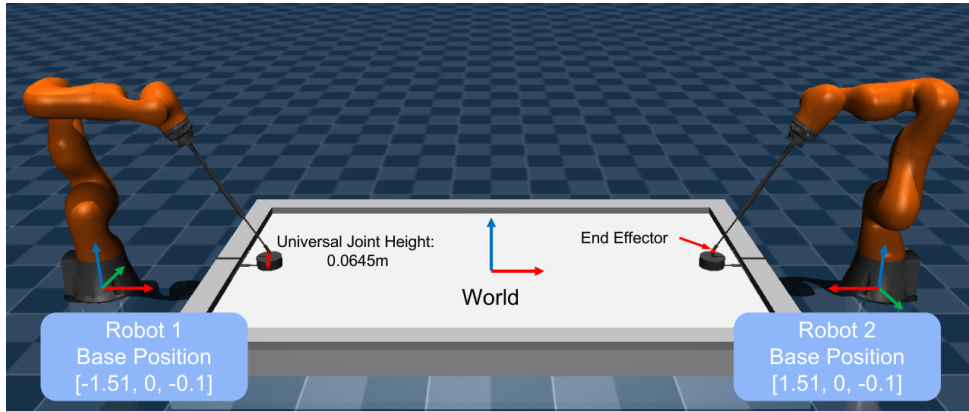


Figure 3.5: KUKA robot base frame.

3.2.3. Constraints

To avoid losing points the robot must respect a set of constraints. These constraints are described here:

- *JointPositionConstraint*: $q_l < q_{cmd} < q_u$
- *JointVelocityConstraint*: $\dot{q}_l < \dot{q}_{cmd} < \dot{q}_u$

- *EndEffectorConstraint*:

$$l_x < x_{ee}$$

$$l_y < y_{ee} < u_y$$

$$\text{table height} - \text{tolerance} < z_{ee} < \text{table height} + \text{tolerance}$$

The End-Effector must not exceed the table and stay at

- *LinkConstraint* (7 DoF Robot only):

$$z_{elbow} > 0.25$$

$$z_{wrist} > 0.25$$

- *ComputationTime*: The computation time at each step should be smaller than 0.02s.

3.2.4. Evaluation Metrics

- **Success Rate**: A success criterion is defined for each task. We will check if the task succeed when the episode terminates. Each episode can terminate because of two reasons: 1. maximum number of steps reached; 2. No further interaction can be in the episodes.
 - Hit: The puck enters the scoring zone with a speed above the threshold.
 - Defend: The final velocity of the puck (at the end of the game) is below the threshold and does not bounce to the other side of the court.
 - Prepare: The final position of the puck is within a predefined area and at a speed less than the threshold.
- **Deployability**: Each deployability metric is assigned one or multiple penalty points based on the level of risk. The deployability score is counted when the constraints of the evaluation metric are violated. Each metric is computed at most once per episode (maximum 500 steps per episode).
 - Violations of the End-Effector's Position Constraints (3 penalty points): The desired x-y-position of the end-effector should remain within the boundaries of the table. The z-position of the end-effector should remain within a range.
 - Violations of the Joint Position Limit Constraints (2 penalty points): The desired position command should not exceed the position limits.
 - Violations of the Joint Velocity Limit Constraints (1 penalty point): The desired velocity command should not exceed the velocity limits.
 - Computation Time (0.5-2 penalty points): The computation time at each step should be shorter than 0.02s.

Each evaluation consisted of 1000 episodes. The success rates were used to rank the leaderboards. The deployability score is the sum of the penalty score from all episodes. The rankings were divided into three categories based on the score of deployability.

1. **Deployable:** $0 \leq \text{deployability score} \leq 500$
2. **Improvable:** $500 < \text{deployability score} \leq 1500$
3. **Nondeployable:** $1500 < \text{deployability score}$

3.2.5. Simulator

The simulator application of the Robot Air Hockey Challenge is summarized in the following:

- Simulator: MuJoCo [20]
- Simulation frequency: 1000 Hz
- Control Frequency: 50 Hz
- Observation:
 - **Robot:**
 - * Joint positions [radians]
 - * Joint velocities [radians/s] computed by finite difference
 - **Puck** (relative to the robot base frame):
 - * X-Y position [m]
 - * Yaw angle [radians]
 - * X-Y velocity [m/s]
 - * Yaw velocity [radians/s] computed by finite difference
 - **Opponent** (if applicable)
 - * End-effector-s X-Y position [m]
- Control action:
 - We try to set the simulation as close as possible to the real-world setup. The robot is controlled by torque at 1000 Hz. We use a feed-forward + PD controller to determine the joint torque that tracks the desired joint position and velocity.
 - A 20-step interpolation is required in the agent's adjacent control commands (the agent control frequency is 50 Hz). We provide different types of polynomial interpolation upon requirements. Details about the action interface can be found here.

Bibliography

- [1] P.-L. Bacon, J. Harb, and D. Precup. The option-critic architecture, 2016.
- [2] P. Dayan and G. E. Hinton. Feudal reinforcement learning. In S. Hanson, J. Cowan, and C. Giles, editors, *Advances in Neural Information Processing Systems*, volume 5. Morgan-Kaufmann, 1992. URL https://proceedings.neurips.cc/paper_files/paper/1992/file/d14220ee66aeec73c49038385428ec4c-Paper.pdf.
- [3] C. D’Eramo, D. Tateo, A. Bonarini, M. Restelli, and J. Peters. Mushroomrl: Simplifying reinforcement learning research. *Journal of Machine Learning Research*, 22 (131):1–5, 2021. URL <http://jmlr.org/papers/v22/18-056.html>.
- [4] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods, 2018.
- [5] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *CoRR*, abs/1801.01290, 2018. URL <http://arxiv.org/abs/1801.01290>.
- [6] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine. Soft actor-critic algorithms and applications. *CoRR*, abs/1812.05905, 2018. URL <http://arxiv.org/abs/1812.05905>.
- [7] H. Hasselt. Double q-learning. In J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 23. Curran Associates, Inc., 2010. URL https://proceedings.neurips.cc/paper_files/paper/2010/file/091d584fced301b442654dd8c23b3fc9-Paper.pdf.
- [8] M. Hutsebaut-Buysse, K. Mets, and S. Latré. Hierarchical reinforcement learning: A survey and open research challenges. *Machine Learning and Knowledge Extraction*, 4(1):172–221, 2022. ISSN 2504-4990. doi: 10.3390/make4010009. URL <https://www.mdpi.com/2504-4990/4/1/9>.
- [9] M. Klissarov, P.-L. Bacon, J. Harb, and D. Precup. Learnings options end-to-end for continuous action tasks, 2017.
- [10] J. Kober, J. A. Bagnell, and J. Peters. Reinforcement learning in robotics: A survey. *International Journal of Robotics Research*, 32(11):1238–1274, 2013. doi: 10.1177/0278364913495721.
- [11] A. D. Laud. *Theory and application of reward shaping in reinforcement learning*. PhD thesis, USA, 2004. AAI3130966.

- [12] P. Liu, D. Tateo, H. Bou-Ammar, and J. Peters. Robot reinforcement learning on the constraint manifold. In A. Faust, D. Hsu, and G. Neumann, editors, *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 1357–1366. PMLR, 2022.
- [13] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller. Playing atari with deep reinforcement learning. *CoRR*, abs/1312.5602, 2013. URL <http://arxiv.org/abs/1312.5602>.
- [14] O. Nachum, S. Gu, H. Lee, and S. Levine. Data-efficient hierarchical reinforcement learning, 2018.
- [15] G. Neumann, W. Maass, and J. Peters. Learning complex motions by sequencing simpler motion templates. In *Proceedings of the 26th Annual International Conference on Machine Learning*, ICML ’09, page 753–760, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605585161. doi: 10.1145/1553374.1553471. URL <https://doi.org/10.1145/1553374.1553471>.
- [16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017.
- [17] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018.
- [18] R. S. Sutton, D. Precup, and S. Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1):181–211, 1999. ISSN 0004-3702. doi: [https://doi.org/10.1016/S0004-3702\(99\)00052-1](https://doi.org/10.1016/S0004-3702(99)00052-1). URL <https://www.sciencedirect.com/science/article/pii/S0004370299000521>.
- [19] R. S. Sutton, J. Modayil, M. Delp, T. Degris, P. M. Pilarski, A. White, and D. Precup. Horde: a scalable real-time architecture for learning knowledge from unsupervised sensorimotor interaction. In *The 10th International Conference on Autonomous Agents and Multiagent Systems - Volume 2*, AAMAS ’11, page 761–768, Richland, SC, 2011. International Foundation for Autonomous Agents and Multiagent Systems. ISBN 0982657161.
- [20] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. doi: 10.1109/IROS.2012.6386109.
- [21] A. S. Vezhnevets, S. Osindero, T. Schaul, N. Heess, M. Jaderberg, D. Silver, and K. Kavukcuoglu. Feudal networks for hierarchical reinforcement learning, 2017.
- [22] C. J. C. H. Watkins. *Learning from Delayed Rewards*. PhD thesis, King’s College, Cambridge, UK, May 1989. URL http://www.cs.rhul.ac.uk/~chrisw/new_thesis.pdf.
- [23] G. Welch, G. Bishop, et al. An introduction to the kalman filter. 1995.